



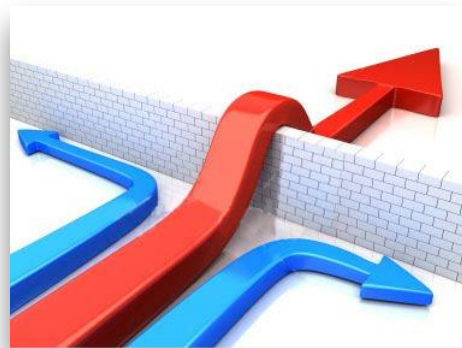
Aprendizado por Reforço

Aulas 9 e 10: Métodos de Monte Carlo

Prof. Charles Madeira

Objetivo da aula

- Apresentar os métodos de Monte Carlo como uma abordagem para o aprendizado por reforço
- Apresentar como os métodos de Monte Carlo calculam funções de valor e identificam políticas ótimas ao avaliar e melhorar políticas
- Mostrar como esses métodos podem ser implementados nas formas on-policy e off-policy



Métodos Monte Carlo

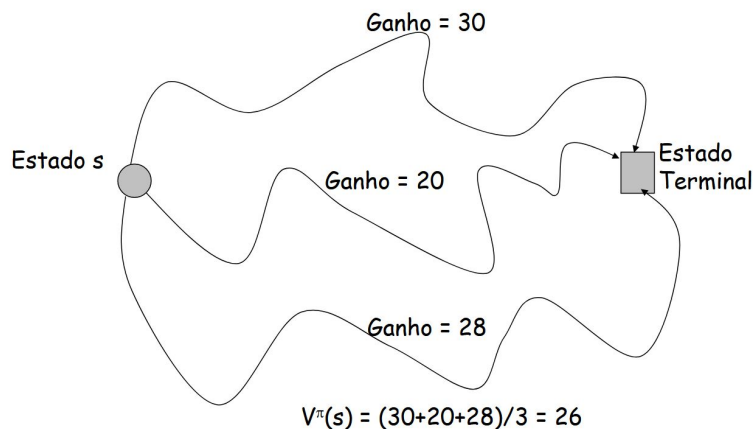
- Os métodos Monte Carlo (MC) se referem a uma classe de métodos estatísticos que se baseiam em amostragem aleatória repetida para obter resultados numéricos
 - Um grande número de amostras repetidas acaba revelando a solução
- Diferentemente dos métodos de Programação Dinâmica (DP), os métodos MC não requerem conhecimento completo do ambiente, se adaptando melhor aos problemas reais
 - A dinâmica do ambiente pode ser desconhecida (*model free*)
 - Fazem uso apenas de experiência amostral (sequência de estados, ações e retornos) a partir da interação com ambientes episódicos

Métodos Monte Carlo

- Aprendizado a partir da experiência on-line é uma excelente alternativa, pois consegue encontrar uma política ótima sem requerer conhecimento a priori da dinâmica do ambiente
 - Necessita gerar apenas exemplos de transições, não sendo necessário distribuição de probabilidades
- Métodos MC são meios de resolver problemas de aprendizado, tomando como base médias de retornos obtidos em sequências
- No sentido de garantir que os retornos sejam bem definidos, métodos MC são definidos como tarefas episódicas completas
 - Assume-se que a experiência é dividida em episódios, que todos os episódios eventualmente terminam, qualquer que seja a sequência de ações
 - As estimativas dos valores e as políticas são modificadas apenas ao final do episódio

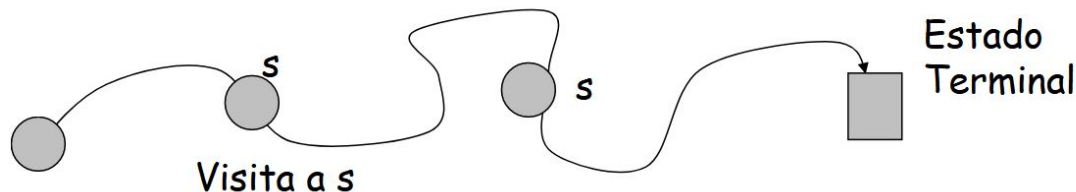
Função de valor da Política Monte Carlo

- Dado uma política π , obter um método de aprendizado para estimar V^π
 - Lembrando que o valor de um estado s é o retorno esperado (ganho futuro, esperado e cumulativo a partir de um estado s)
 - Logo, uma forma de estimar o valor de um estado s é calcular a média dos retornos observados após visitá-lo



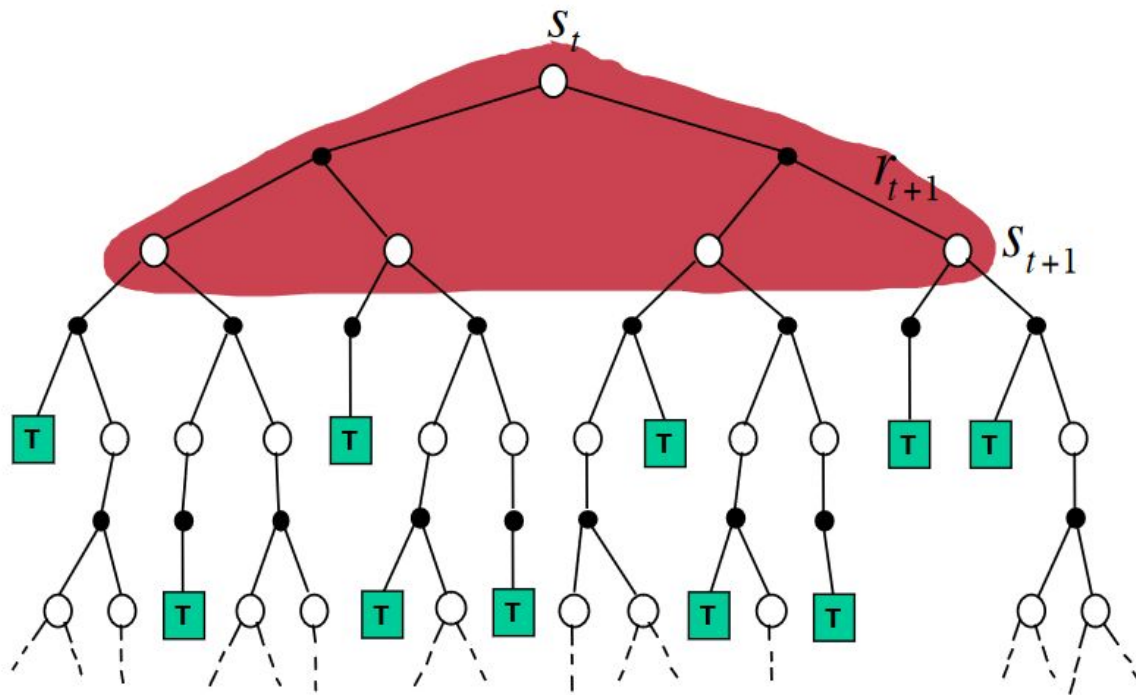
Função de valor da Política Monte Carlo

- A ideia básica para estimar $V^\pi(s)$, que é o valor de um estado s sob a política π , consiste em calcular a média dos retornos observados após visitar o estado s , dado um conjunto de episódios obtidos a partir de π e passando por s
 - Cada ocorrência de s em um episódio é chamada de visita a s
 - À medida que trajetórias são produzidas e retornos são observados, a média deve convergir para o valor esperado



Backup da Programação Dinâmica

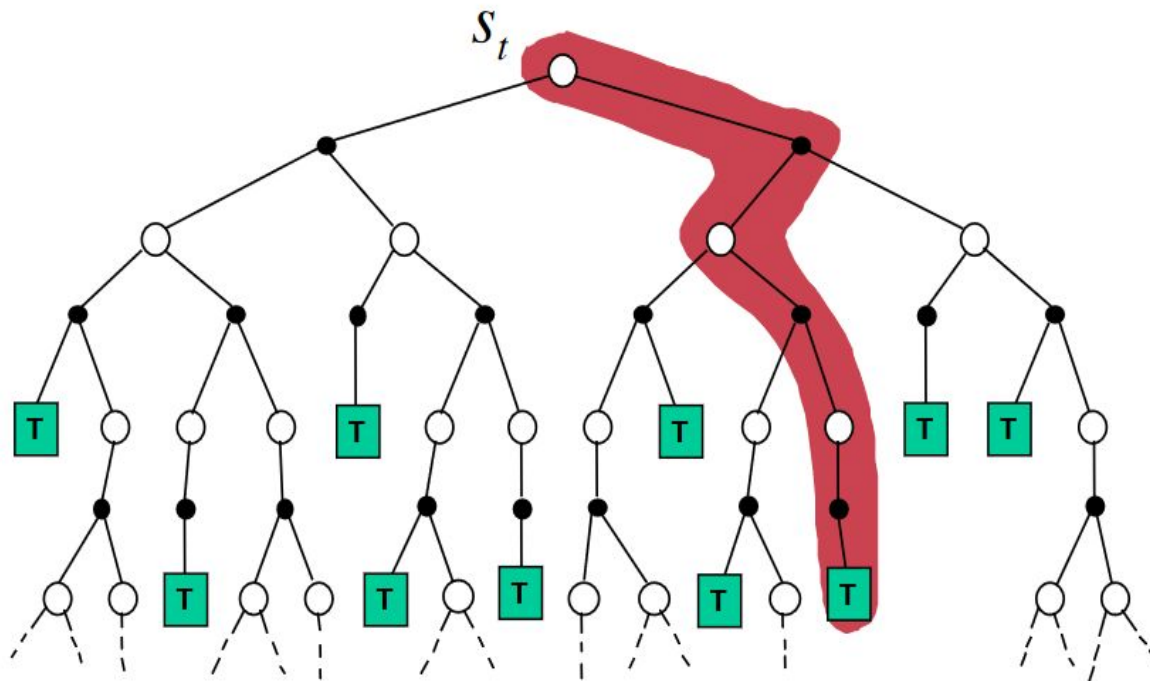
$$V(S_t) \leftarrow \mathbb{E}_{\pi} [R_{t+1} + \gamma V(S_{t+1})]$$



Backup do Monte Carlo

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^T \gamma^k r_{t+k+1}$$

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

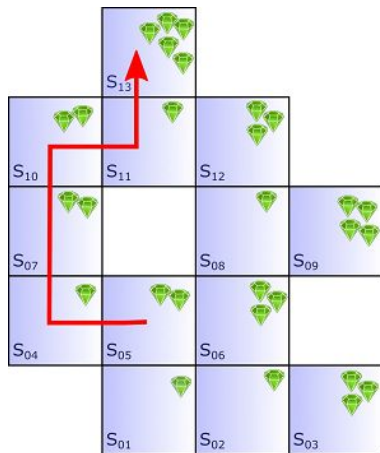


Tipos de métodos de Predição Monte Carlo

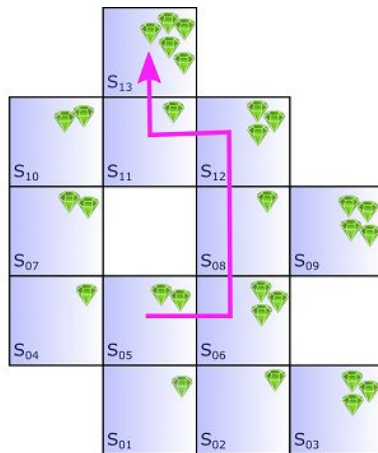
- **Método de Primeira Visita (*First-visit*):** estima $V^\pi(s)$ como a média dos retornos que seguem a primeira visita a um estado s em cada episódio
 - Na primeira vez que s é visitado no episódio, incrementar o contador $N(s) = N(s) + 1$
 - Incrementar o retorno total com o retorno estimado no episódio $TR(s) = TR(s) + G_t$
 - A função valor é estimada pela média dos retornos $V(s) = TR(s) / N(s)$
- **Método de Toda Visita (*Every-visit*):** estima $V^\pi(s)$ como a média dos retornos que seguem a todas as visitas a um estado s em cada episódio
 - A cada vez que s é visitado no episódio, incrementar o contador $N(s) = N(s) + 1$
 - Incrementar o retorno total com o retorno estimado no episódio $TR(s) = TR(s) + G_t$
 - A função valor é estimada pela média dos retornos $V(s) = TR(s) / N(s)$
- Ambos os métodos convergem para $V^\pi(s)$ à medida que o número de visitas (ou primeiras visitas) tende ao infinito
 - Pela lei dos grandes números, $V(s) \rightarrow V^\pi(s)$, assim como $N(s) \rightarrow \infty$

Exemplo do cálculo da função valor

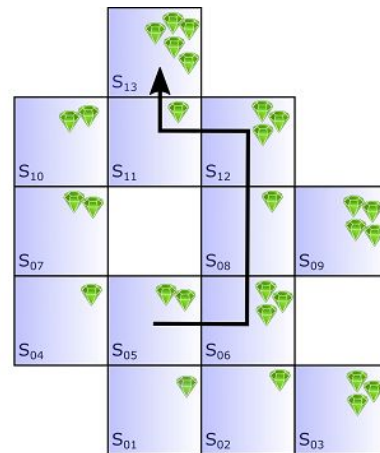
- Cenário no qual um agente deve ir do estado S_{05} para o estado S_{13} coletando jóias



Episódio 1



Episódio 2

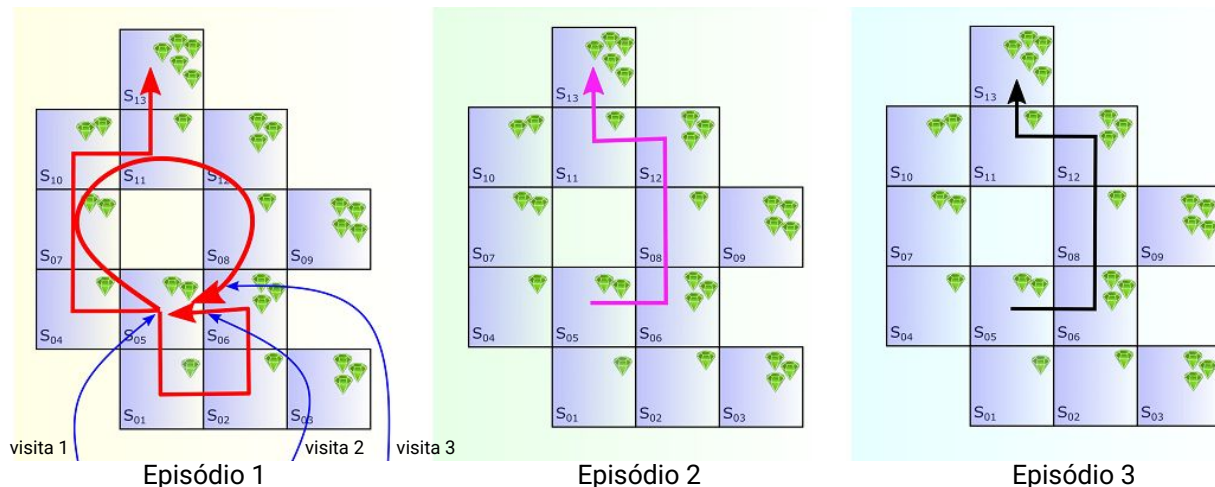


Episódio 3

- $N(S_{05}) = 1 + 1 + 1 = 3$
- Episódio 1: $TR(S_{05}) = 2 + 1 + 2 + 2 + 1 + 5 = 13$ jóias
- Episódio 2: $TR(S_{05}) = 2 + 3 + 1 + 3 + 1 + 5 = 15$ jóias
- Episódio 3: $TR(S_{05}) = 2 + 3 + 1 + 3 + 1 + 5 = 15$ jóias
- $V(S_{05}) = (TR(S_{05})_1 + TR(S_{05})_2 + TR(S_{05})_3) / N(S_{05}) = (13 + 15 + 15) / 3 = 14,33$ jóias

Exemplo do cálculo da função valor (*First-visit*)

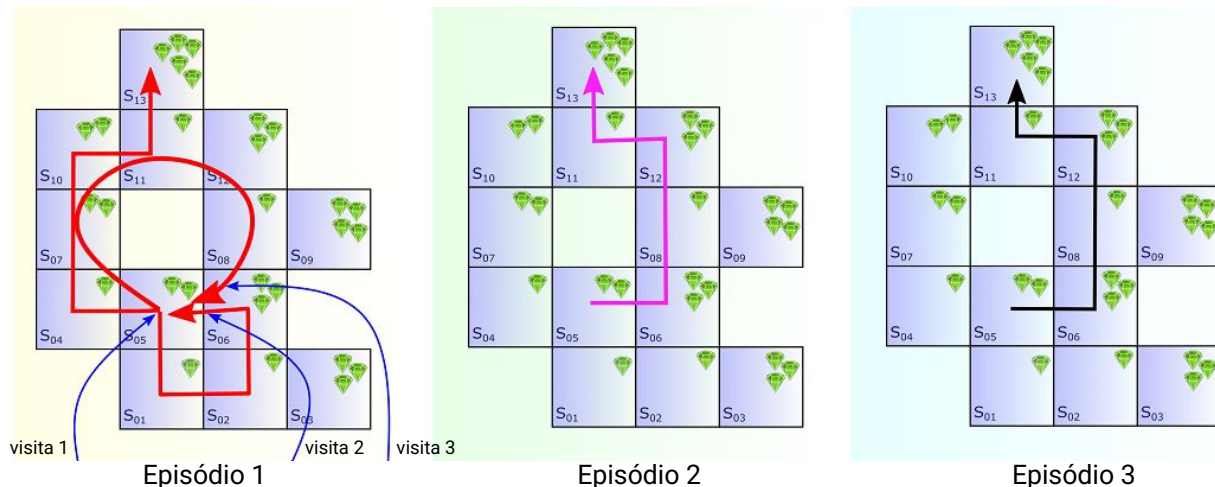
- Mesmo que o agente volte ao mesmo estado, apenas a primeira visita é contada



- $N(S_{05}) = 1 + 1 + 1 = 3$
- Episódio 1: $TR(S_{05}) = 13 + 15 + 7 = 35$ jóias
- Episódio 2: $TR(S_{05}) = 15$ jóias
- Episódio 3: $TR(S_{05}) = 15$ jóias
- $V(S_{05}) = (35 + 15 + 15) / 3 = 21,66$ jóias

Exemplo do cálculo da função valor (*Every-visit*)

- Cada visita é contada para um mesmo estado no episódio



- $N(s_{05}) = 3 + 1 + 1 = 5$
- Episódio 1: $TR(s_{05}) = 13 + 15 + 7 = 35$ jóias
- Episódio 2: $TR(s_{05}) = 15$ jóias
- Episódio 3: $TR(s_{05}) = 15$ jóias
- $V(s_{05}) = (35 + 15 + 15) / 5 = 13$ jóias

Método MC *First-visit*

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

s	$V(s)$
s_0	R_t
s_1	$\dots$
$\dots$	$\dots$
s_n	$\dots$

Análise do Blackjack

- Jogo cujo objetivo do jogador é obter 2 cartas do baralho em que a soma de seus valores deve ser o mais próximo possível de 21, mas sem exceder esse valor
 - As cartas com figuras valem 10 e o Ás pode valer 1 ou 11
- O jogador e o distribuidor contêm ambos 2 cartas, mas apenas 1 das cartas do distribuidor fica à mostra na mesa
- Forma de jogo
 - O jogador pode pedir uma nova carta (*hit*) até decidir parar (*stick*) ou ultrapassar o valor de 21 (*goes bust*)
 - O distribuidor segue uma estratégia fixa, pedindo carta até que a soma seja igual ou superior a 17
- As cartas compõem um baralho de tamanho infinito, não havendo vantagem em memorizar cartas retiradas da mesa



Análise do Blackjack

- Quando o modelo é conhecido, os valores dos estados são suficientes, pois simplesmente analisamos um passo à frente e escolhemos a ação que nos leva ao próximo estado com o melhor retorno
 - A Programação dinâmica funciona bem
- Mas neste tipo de problema do Blackjack, seria muito difícil aplicar métodos da programação dinâmica para calcular a função valor
 - A programação dinâmica requer uma distribuição para os próximos estados, com uma função de probabilidade de transição que não é de cálculo simples

$$V^*(s) = \max_{\pi} V^{\pi}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Modelagem do Blackjack em MDP

- Blackjack é naturalmente formulado como um MDP episódico e finito
 - Cada partida consiste em um episódio
- Espaço de estados **S** do ambiente = configuração das cartas
 - Soma das cartas do jogador
 - Carta aparente do distribuidor
 - Disponibilidade de Ás
- Espaço de ações **A** do agente = {Hit, Stick}
 - *Hit*: Recebe uma nova carta
 - *Stick*: Para de receber cartas
- Função de probabilidade de transição **T**
 - Não é exigida pelo método Monte Carlo (*free model*)
- Função de reforço **R**
 - Stick
 - +1: se soma das cartas do jogador é 21 e maior que a soma das cartas do distribuidor
 - 0: se soma das cartas do jogador é 21 e igual a soma das cartas do distribuidor
 - -1: se soma das cartas do jogador é menor que a soma das cartas do distribuidor
 - Hit
 - -1: se soma das cartas do jogador > 21 (término)
 - 0: caso contrário

Estimativa de valores das ações

- Quando um modelo não está disponível e o valor do estado sozinho não se mostrar suficiente, será útil estimar explicitamente o valor das ações $Q^*(s,a)$ ao invés do valor dos estados $V^*(s)$
 - Isso permite sugerir uma política de controle
- Um dos objetivos principais dos métodos MC é estimar $Q^*(s,a)$, a única complicação estando relacionada ao fato que muitos pares estado-ação podem não ser visitados
 - Se π é uma política determinística, então ao seguir π observa-se retornos de apenas uma ação para cada estado
- Para que os métodos MC funcionem, é preciso estimar o valor de todas as ações a partir de cada estado, ou seja, não apenas daquelas ações favorecidas pela política
 - Problema geral de manter a propriedade de exploração

Dilema de explorar ou usufruir

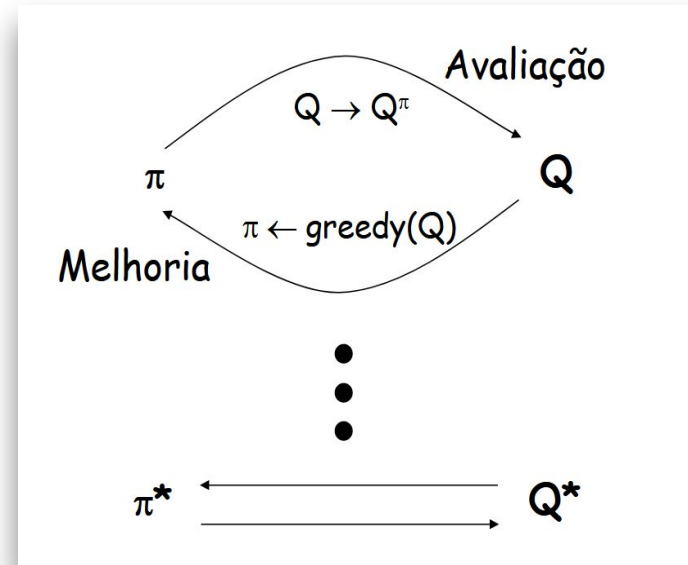
- Usufruir (*exploitation*)
 - Escolher a ação que está atualmente com o maior valor $Q(s,a)$ na memória
- Explorar (*exploration*)
 - Escolher uma ação aleatória para que seu valor $Q(s,a)$ seja atualizado
- Dilema
 - Dado que aprendeu que $Q(s,a)$ vale x , vale a pena tentar executar a ação a' , uma vez que atualmente $Q(s,a')$ vale $x/5$?
 - Depende do ambiente, da quantidade de ações já tomadas e da quantidade de ações restantes

Dilema de explorar ou usufruir

- Duas maneiras de garantir a propriedade de exploração
 - Iniciar cada episódio em um par estado-ação, sendo que cada um tenha propriedade não nula de ser selecionado como inicial
 - Considerar apenas políticas que associam probabilidade não nula a todas as ações
- Posição que permite considerar como a estimativa Monte Carlo pode ser utilizada no controle, ou seja, na aproximação da política ótima
 - Procede-se da mesma forma que em GPI (*Generalized Policy Iteration*), mantendo aproximações para a política e para a função valor
 - Para métodos MC, é natural alternar entre [avaliação de política](#) e [melhoria de política](#) de episódio em episódio

Interação entre a política e a função valor

- As duas formas (**avaliação** e **melhoria**) trabalham, de certa forma, uma contra a outra
- Cada forma define uma meta para a outra, mas juntamente fazem com que a política e a função valor estado-ação se aproximem do ótimo
- A iteração de política clássica Monte Carlo assume um **número infinito de episódios** para cada avaliação de política e partidas exploratórias
- A melhoria da política é feita tornando a **política gananciosa** (ϵ -greedy) em relação à função de valor de ação atual: $\pi(s) \leftarrow \operatorname{argmax}_a Q(s,a)$



Método de Controle MC ES (*Exploring Starts*)

- Método que remove a suposição do número infinito de episódios para avaliação da política, alternando avaliação e melhoria episódio a episódio
 - Após cada episódio, os retornos observados são usados para avaliação da política
 - A política é melhorada em todos os estados visitados no episódio
 - A estabilidade é alcançada apenas quando a política e a função de valor estado-ação são ótimas
- Método de fácil aplicação no Blackjack, pois as partidas exploratórias (escolher cartas do distribuidor, soma do jogador, e se há um Ás usável aleatoriamente) são facilmente arranjadas

Método de Controle MC ES (*Exploring Starts*)

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

s	a	$Q(s, a)$
s_0	a_0	R_t
s_0	a_1	$\dots$
$\dots$	$\dots$	$\dots$
s_n	a_0	$\dots$
s_n	a_1	$\dots$

Monte Carlo *Off-policy* vs *On-Policy*

- Podemos evitar a condição irrealista de partidas exploratórias?
 - A única forma geral para garantir que as ações sejam selecionadas infinitamente é fazer o agente selecioná-las continuamente
- Para isso, existem duas abordagens possíveis
 - Métodos *off-policy*
 - Adotam uma política para gerar o comportamento do agente (***behavior policy***) e uma outra política para ser avaliada e melhorada (***estimation policy***)
 - A vantagem desta separação é que a política estimada pode ser gananciosa (*ϵ -greedy*), enquanto que a política de comportamento pode selecionar todas as ações
 - Métodos *on-policy*
 - Adotam uma única política que é estimada e utilizada para controle, sendo avaliada e melhorada na tomada de decisão

Monte Carlo On-Policy

- No método MC **On-Policy**, a política é geralmente flexível (ϵ -soft)
 - $\pi(s,a) > 0$ para todo $s \in S$ e $a \in A(s)$
- Há várias possibilidades de se implementar políticas flexíveis
 - Uma possibilidade é tender a política gradualmente para uma política determinística ótima (gananciosa)
 - Quase sempre escolher uma ação com máximo retorno estimado, mas com probabilidade não nula de escolher outras ações aleatoriamente
 - As ações gananciosas recebem probabilidade igual a $\epsilon/|A(s)|$
 - As outras ações recebem probabilidade igual a $1 - \epsilon + \epsilon/|A(s)|$

Control Monte Carlo On-Policy

On-policy first-visit MC control (for ε -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

s	a	$Q(s,a)$
s_0	a_0	R_t
s_0	a_1	$\dots$
$\dots$	$\dots$	$\dots$
s_n	a_0	$\dots$
s_n	a_1	$\dots$

Vantagens e limitações do Monte Carlo

- Vantagens

- **Permite aprendizado sem modelo:** pode aprender comportamento ótimo diretamente da interação com o ambiente, sem a necessidade de um modelo da dinâmica do ambiente
- **Faz uso de modelos de amostra:** pode ser usado com simulações ou modelos de amostra, que é fácil simular episódios, mas difícil construir um modelo explícito de probabilidades de transição
- **Foca em subconjuntos de estados:** é eficiente para focar em um pequeno subconjunto de estados, avaliando-o com precisão sem a necessidade de avaliar o resto do conjunto de estados
- **É menos afetado por violações da propriedade de Markov:** por não fazer bootstrapping, pode ser menos prejudicado por violações da propriedade de Markov, pois não atualiza suas estimativas de valor com base em estimativas de valor de estados sucessores

- Limitações

- Deve esperar até o final do episódio para que o valor de retorno seja conhecido
- Tem alta variância
- Só consegue aprender com sequências completas
- Só funciona no contexto de ambientes episódicos

Exercício

- Experimentar os métodos Monte Carlo no Blackjack
- Usar como base a implementação do [Drive](#)

